Федеральное государственное автономное образовательное учреждение
высшего образования

**«Национальный исследовательский университет
“Высшая школа экономики”»**

Факультет компьютерных наук
Департамент программной инженерии

Отчёт по индивидуальному заданию № 2

по дисциплине «Архитектура компьютера и операционные системы»

Вариант 14. «Задача об умных цветах»

Многопоточная имитационная программа

Выполнила: Гончарова Екатерина
Группа: БПИ-253

Москва, 2026

Содержание

1 Постановка задачи **2**

2 Соответствие сущностей потокам **2**
2.1 Поток цветка 2
2.2 Поток садовника 2
2.3 Главный поток 2

3 Синхронизация и общая память **3**
3.1 Мьютекс состояния 3
3.2 Семафоры и уведомления 3
3.3 Потокобезопасный вывод 3

4 Входные параметры и пользовательский интерфейс **3**

5 Корректное завершение **4**

6 Проверка программы **4**
6.1 Тестовые сценарии 4
6.2 Инварианты 5

7 Особенности реализации **5**

8 Итог **5**

1 Постановка задачи

Требуется разработать консольное приложение на языке C для ОС Linux, имитирующее взаимодействие умных цветов и садовников с помощью **независимых потоков исполнения**. По условию варианта 14 каждый цветок самостоятельно изменяет свою влажность, определяет необходимость полива и посылает заявку. Садовники реагируют на заявки и конкурируют за право их исполнения. Для одной заявки может быть назначен только один садовник. Повторный полив в течение защитного интервала вызывает перелив.

Состояния цветка: нормальное развитие (`NORMAL`), увядание (`WILTING`), полив (`WATERING`), восстановление (`RECOVERING`), засыхание (`DRY`), гибель от перелива (`DEAD_OVERWATERED`). Приложение обязано отображать смену состояний, создание и снятие заявки, её принятие садовником, конкуренцию за заявки, полив, восстановление, засыхание и перелив.

Для реализации используются `pthread`, мьютексы и семафоры. Каждый участник модели выполняет собственный цикл обработки событий.

2 Соответствие сущностей потокам

Создаются N потоков цветов и M потоков садовников, то есть всего $N + M$ рабочих потоков. Главный поток используется только для создания и завершения рабочих потоков, отслеживания времени и вывода итоговых данных.

2.1 Поток цветка

Каждый поток получает указатель на общую модель и индекс только своего цветка. Через условный такт он:

1. захватывает мьютекс модели;
2. изменяет влажность и состояние своего цветка;
3. при достижении порога создаёт одну активную заявку;
4. отправляет уведомление через семафоры садовникам;
5. освобождает мьютекс и ждёт следующий такт.

Поток не создаёт новую заявку до завершения или отмены предыдущей. Засохший либо погибший цветок больше не меняется и не подаёт заявки.

2.2 Поток садовника

Каждый садовник ждёт на собственном семафоре и не загружает процессор циклом опроса. После получения уведомления он под мьютексом выбирает свободную заявку по правилу `first` или `driest`. При успешном назначении запоминает индекс цветка и начинает работать. При отсутствии свободной заявки, если другая заявка уже принята, фиксируется конфликт. Два потока, конкурирующие за заявку, не могут успешно назначить её одновременно: проверка доступности и назначение выполняются в критической секции.

Движение садовника и полив занимают заданное число шагов времени. Садовник спит между этими шагами, не удерживая мьютекс. Во время подхода цветок может окончательно засохнуть; садовник тогда прекращает обслуживание. При завершении полива восстановление влажности и снятие заявки выполняются атомарно.

2.3 Главный поток

Главный поток создаёт необходимые объекты синхронизации и рабочие потоки с помощью `pthread_create()`, после чего ожидает истечения заданного периода наблюдения либо

прекращения активности всех цветов. Он завершает работу при Ctrl+C и ожидает каждый запущенный поток через pthread_join().

3 Синхронизация и общая память

3.1 Мьютекс состояния

model_mutex защищает массив цветов, массив садовников, признаки активных заявок и назначенных садовников, статистические счётчики и флаг завершения. Ни один рабочий поток не читает и не изменяет эти общие переменные вне критической секции. В частности, принятие заявки осуществляется как неделимая операция.

Упрощённая схема выбора и назначения заявки:

```
pthread_mutex_lock(&simulation->model_mutex);
update_gardener(gardener, simulation->flowers,
                simulation->settings, simulation->current_tick);
pthread_mutex_unlock(&simulation->model_mutex);
```

Вызов update_gardener при свободном садовнике находит доступную заявку, проверяет request_active, assigned_gardener и состояние цветка, после чего закрепляет её за одним садовником.

3.2 Семафоры и уведомления

У каждого садовника есть свой семафор уведомлений. Когда появляется заявка, поток цветка вызывает sem_post() для каждого садовника. Свободный садовник ожидает сигнал с помощью sem_wait(). После пробуждения он пытается принять заявку. Таким образом два садовника могут реагировать на одну и ту же заявку, но только один получает право на полив.

Уведомление не эквивалентно гарантированному назначению: заявка к моменту пробуждения может быть уже принята или отменена. Накопившиеся уведомления безопасны, так как реальное состояние всегда проверяется под мьютексом.

3.3 Потокобезопасный вывод

log_mutex сериализует запись событий в стандартный вывод и simulation.log. Используются open(), write() и close(); вспомогательная функция повторяет запись при частичном write() и при EINTR. Это предотвращает перемешивание фрагментов разных сообщений.

4 Входные параметры и пользовательский интерфейс

Интерфейс поддерживает интерактивный ввод и запуск с файлом параметров. Программа принимает десять исходных параметров, соответствующих условию: количество цветов, массив индивидуальных скоростей высыхания, порог заявки, порог засыхания, длительность полива, восстановления, защитного интервала, число садовников, скорость их работы и правило выбора заявки.

Возможны два способа запуска: интерактивный, без аргументов, и по конфигурационному файлу key=value. Например:

```
make
```

```
./build/smart_flowers
./build/smart_flowers tests/balanced.cfg
make check
```

Модель использует 50 условных тактов с паузой 25 миллисекунд между шагами каждого потока. Длительность такта и число тактов определены именованными константами `TICK_DURATION_MS` и `OBSERVATION_PERIOD`. Это параметры визуализации и границы наблюдения; они не изменяют формат входных конфигураций.

5 Корректное завершение

Программа заканчивается после 50 условных тактов, при прекращении активности всех цветов либо при получении `SIGINT`. Потоки наследуют заблокированный `SIGINT`; поэтому сигнал обрабатывается главным потоком. Обработчик меняет только `sig_atomic_t`. Главный поток устанавливает защищённый мьютексом флаг остановки, пробуждает ожидающих садовников, дожидается завершения всех потоков, печатает статистику и удаляет примитивы синхронизации.

Незаконченный к моменту остановки полив не включается в число *успешно завершённых* поливов. В итоговой статистике выводятся число живых, засохших и погибших от перелива цветов, влажность каждого цветка и число завершённых поливов. В журнале дополнительно выводятся количество созданных заявок и конфликтов.

6 Проверка программы

6.1 Тестовые сценарии

Предусмотрено десять конфигураций с разными сочетаниями числа потоков, порогов, длительности полива и восстановления, скорости садовников и правил выбора заявок:

- `balanced.cfg`: обычная работа двух садовников;
- `driest_policy.cfg`: выбор наиболее сухого цветка;
- `single_gardener.cfg`: очередь при одном садовнике;
- `overwatering.cfg`: проверка гибели от перелива;
- `fast_drying.cfg`: быстрое засыхание;
- `many_gardeners.cfg`: конкуренция нескольких садовников;
- `large_garden.cfg`: 25 цветов и 10 садовников;
- `long_recovery.cfg`: длительное восстановление;
- `no_protection.cfg`: нулевой защитный интервал;
- `one_flower.cfg`: граничный случай с одним цветком.

Команда `make check` запускает все десять конфигураций с помощью небольшого скрипта на Python. Скрипт проверяет код завершения программы и наличие итоговой статистики. Для `overwatering.cfg` дополнительно проверяется, что хотя бы один цветок погиб от перелива.

Эти проверки помогают находить ошибки запуска и очевидные нарушения логики моделирования. Они не проверяют все возможные порядки выполнения потоков. При ручной проверке можно запустить отдельную конфигурацию и посмотреть журнал `simulation.log`. Порядок событий в нём может отличаться между запусками.

6.2 Инварианты

При работе программы должны соблюдаться следующие условия:

1. одну активную заявку получает не более одного садовника;
2. каждый садовник обслуживает не более одного цветка;
3. один цветок имеет не более одной активной заявки;
4. закрытая или отменённая заявка не может быть снова обслужена;
5. погибший цветок не формирует заявки и не продолжает активную работу.

7 Особенности реализации

Каждый цветок и садовник работает в отдельном потоке. Для доступа к общему состоянию используются мьютексы, для уведомлений — семафоры. Программа использует один мьютекс для состояния модели. Это упрощает назначение заявок: проверка и закрепление выполняются вместе.

Общий мьютекс упрощает код, но при большом количестве потоков может ограничивать скорость работы. Перемещение садовника задаётся количеством работы, а не координатами. Также у потоков нет общего барьера в конце каждого такта: они выполняют шаги независимо друг от друга. Поэтому записи с одинаковым номером такта не обязательно относятся к одному моменту времени.

8 Итог

Разработана многопоточная модель умных цветов и садовников с конфигурационным вводом, журналом и итоговой статистикой. Каждый цветок и каждый садовник имеет отдельный поток, а общий доступ синхронизируется мьютексами и семафорами. Предусмотрено нормальное и досрочное завершение, ведение журнала и автоматизированная проверка.